

Curso de Programación en Java

Nivel I

Introducción a la POO

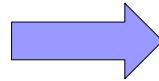
Clases, Objetos, Métodos y Atributos

Codificación en el Lenguaje

La complejidad del Software

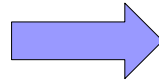
Es una propiedad esencial del software, depende 4 partes:

- El *dominio* del problema



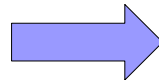
Requisitos muy distintos, cambiantes,
contradictorios.

- El *proceso* de desarrollo



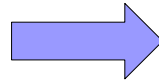
Intervienen muchos desarrolladores, se necesita coordinar y utilizar una tecnología en común.

- La *flexibilidad* en el software

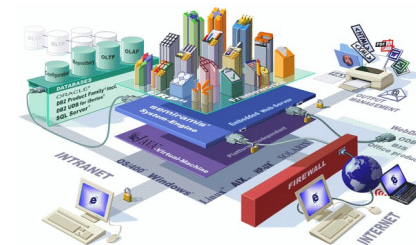


Sistema apto para cambiar con los requerimientos del cliente

- La *reutilización* del código

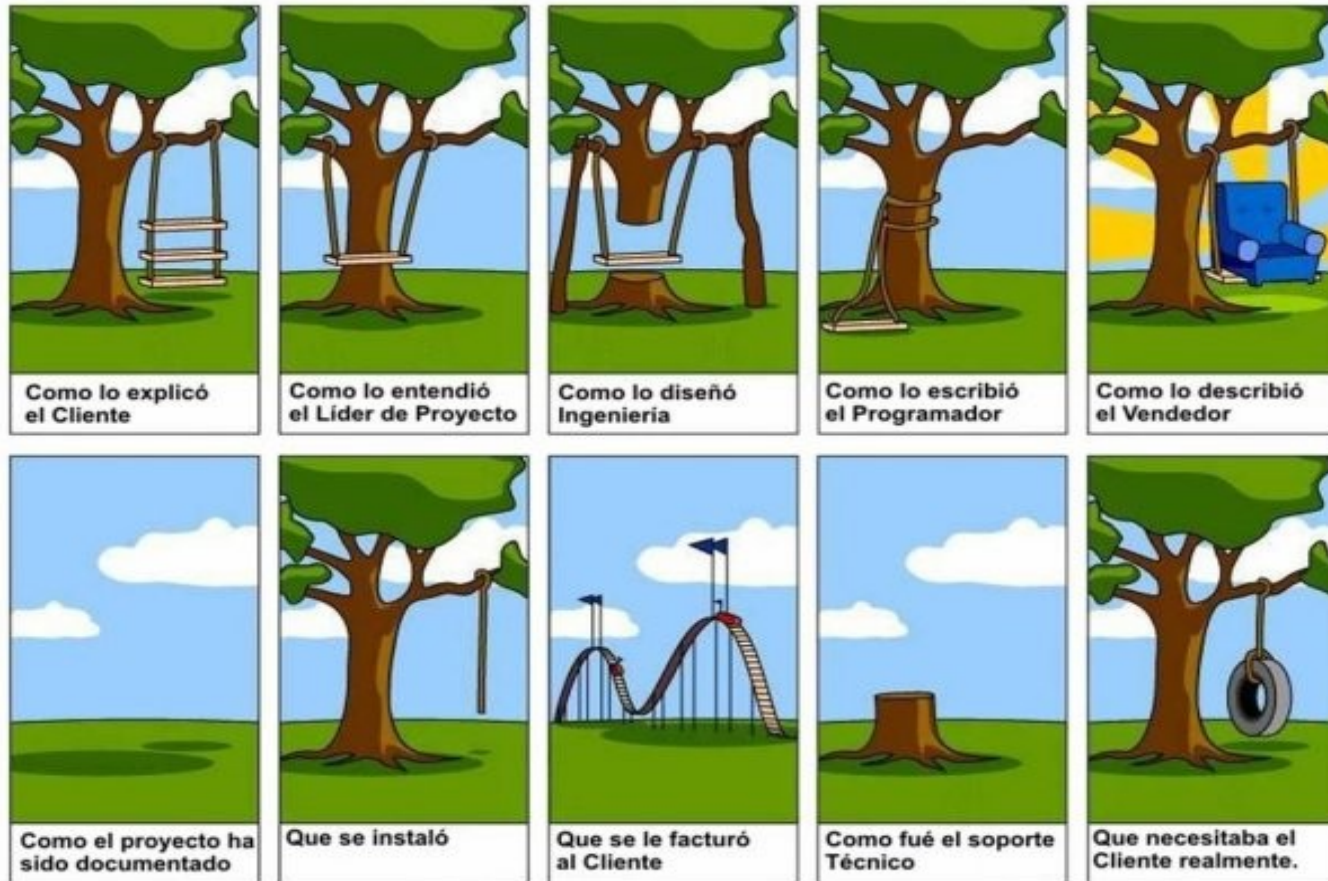


No reinventar la rueda.



La complejidad del Software

Una triste realidad de los proyectos de Software...



Programación Orientada a Objetos

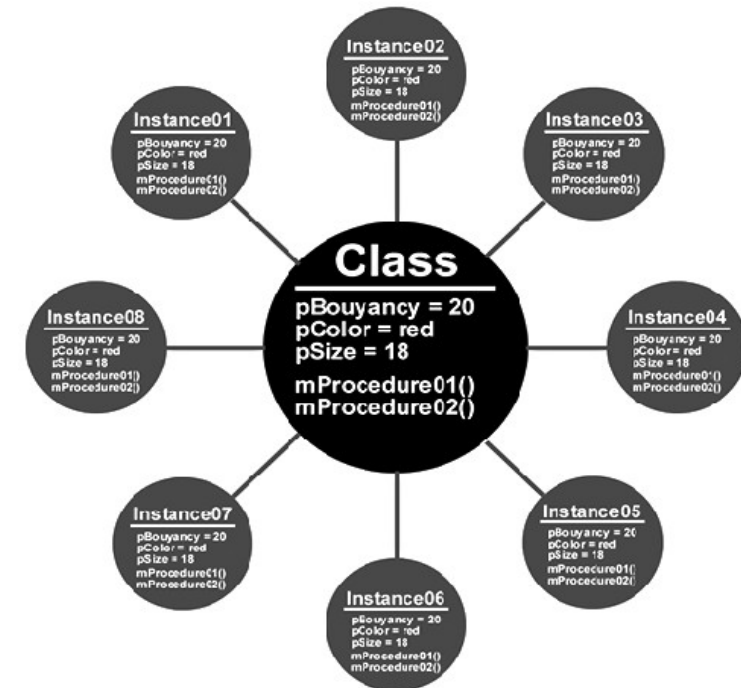
Es un paradigma de programación, es decir una forma de *analizar, diseñar y realizar soluciones*.

Objetivos:

- Escribir software fácilmente modificable y escalable.
- Escribir software reusable.
- Disponer de un modelo natural para representar un dominio.

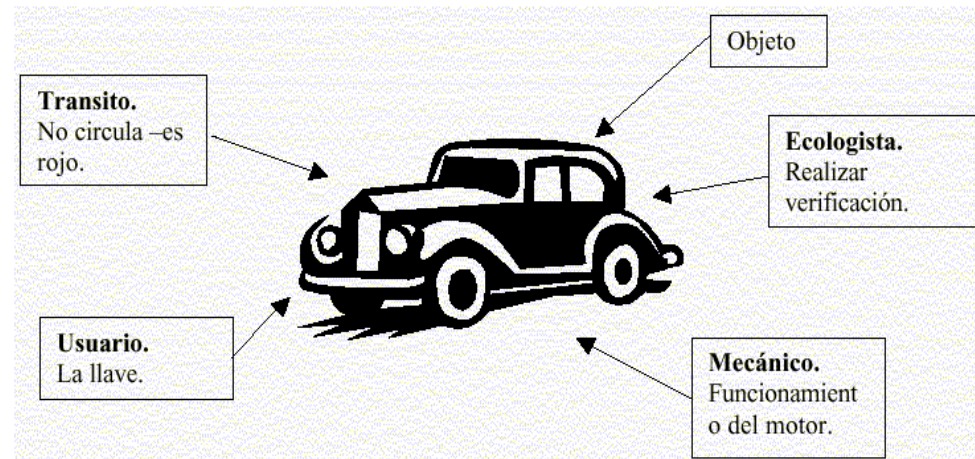
Ventajas:

- Fomenta la reutilización del software.
- El software desarrollado es más flexible al cambio.
- Es más cercano a pensar a la forma de las personas.



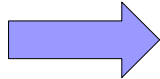
¿Qué es la abstracción?

- Consiste en aislar un elemento de su contexto o del resto de los elementos que lo acompañan.
- Denota según el **observador** las características esenciales de un objeto que lo distinguen de los demás.
- La vista exterior del objeto puede definirse como un contrato, el cual el mismo se compromete a cumplir y los demás objetos dependen.
- Debemos identificar propiedades del objeto (*atributos*) y comportamientos (*métodos*).



Objetos y Clases

Clases



- Modela una abstracción de los objetos.
- Define atributos y comportamientos.
- Es el plano o molde que define un objeto.

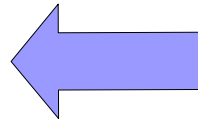
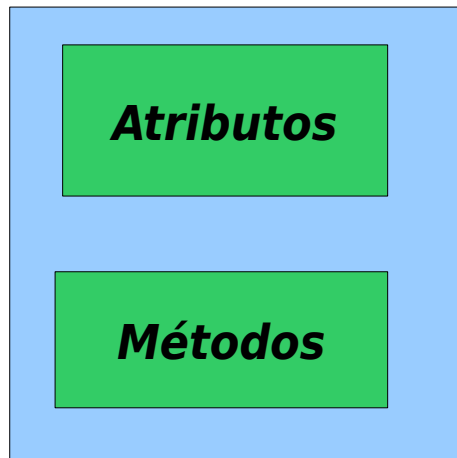
Objetos



- Está modelado *en función* de una clase.
- Es una *instancia* única de una clase.
- Retiene la estructura y el comportamiento de una clase.

¿Qué es un Objeto?

- Es una entidad que combina procedimientos e información.
- Ejecuta operaciones (***comportamiento***) y almacena información (***estructura***).
- Los procedimientos o acciones que puede realizar un objeto se denominan ***métodos***.
- La información que almacena un objeto sobre su estado se denomina ***atributos***.



Objeto

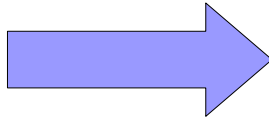
Los datos describen el *estado del objeto*.

Los procedimientos actúan sobre los datos del objeto, *modificándolos o brindando* información

¿Qué es un Objeto?

Un objeto posee...

Estado



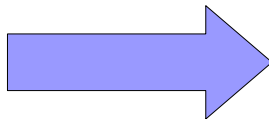
Esta dado por el conjunto de propiedades que posee y su valor en un momento dado.

Comportamiento



Está dado por un conjunto de acciones que el mismo puede realizar.

Identidad

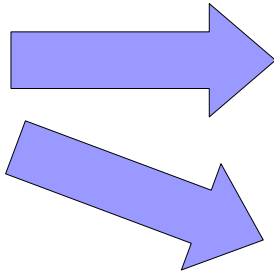


En todo momento es diferenciable del resto y es constante.

Estado de un Objeto

- Está dado por el conjunto de propiedades que el mismo posee.
- Es deseable encapsular el estado de un objeto, ocultando su representación al mundo exterior.
- Ningún objeto puede manipular directamente el estado de otro.

Propiedades

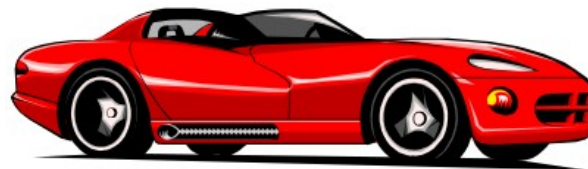


Estáticas: propiedad compartida por todos los objetos del mismo tipo.

Dinámicas: propiedad inherente de cada objeto

Comportamiento de un Objeto

- Determina como éste actúa y reacciona en términos de cambios de estado y solicitudes.
- Representan la interfaz por la cual se puede interactuar con el objeto.
- Los métodos asociados a un objeto comprenden el protocolo del mismo.



■ Atributos:

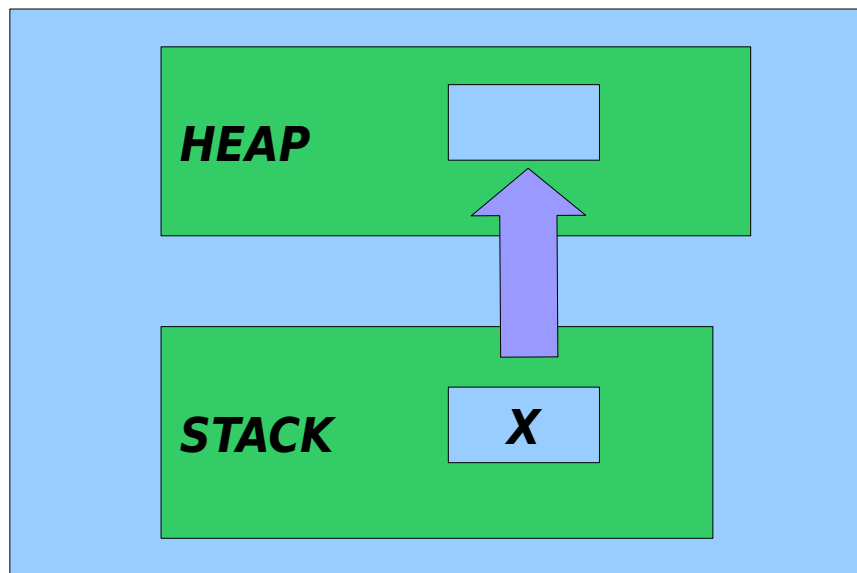
- color
- velocidad
- ruedas
- motor

■ Métodos:

- arranca()
- frena()
- dobla()

Identidad de un Objeto

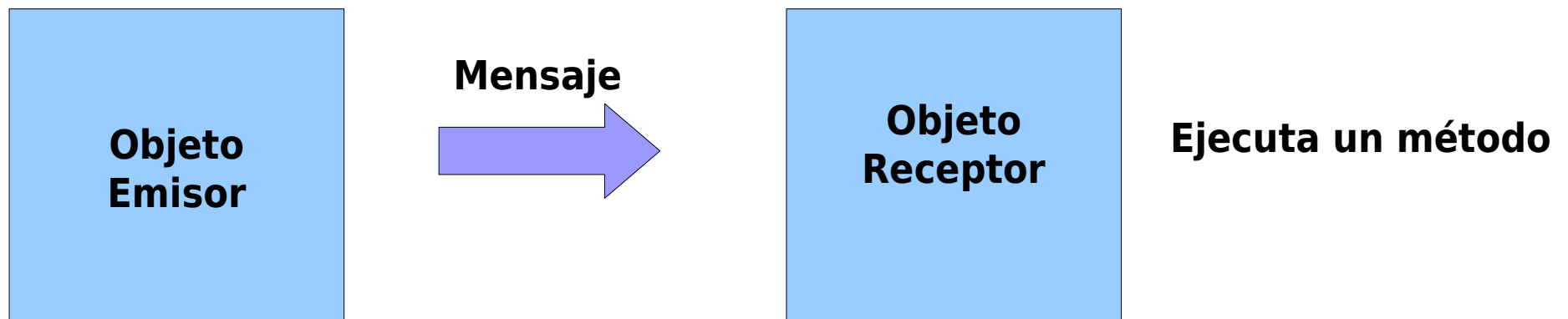
- Es una propiedad del objeto que lo diferencia del resto.
- No debe confundirse con el estado.
- Es preservada en el tiempo aún cuando su estado cambie.
- En muchos lenguajes de programación la identidad está determinada por la dirección de memoria en la que el mismo se encuentra.



X = 0xFFEE1234
Identidad

Los objetos interactúan a través de mensajes

- Los objetos se comunican mediante el envío de mensajes.
- El objeto emisor se debe ***enlazar*** o ***asociar*** al objeto receptor.
- El emisor del mensaje solicita al receptor que realice una operación.
- La llamada al método siempre se produce en el contexto de un objeto concreto.



¿Qué es una Clase?

- Mecanismo utilizado en la POO para abstraer conceptos (clasificación).
- Describe las características comunes a todos los objetos que pertenecen a ella.
- Especifica:
 - ✓ El ***comportamiento*** de los objetos.
 - ✓ La ***estructura interna*** de los objetos.
 - ✓ La definición e implementación de todas sus ***acciones***.

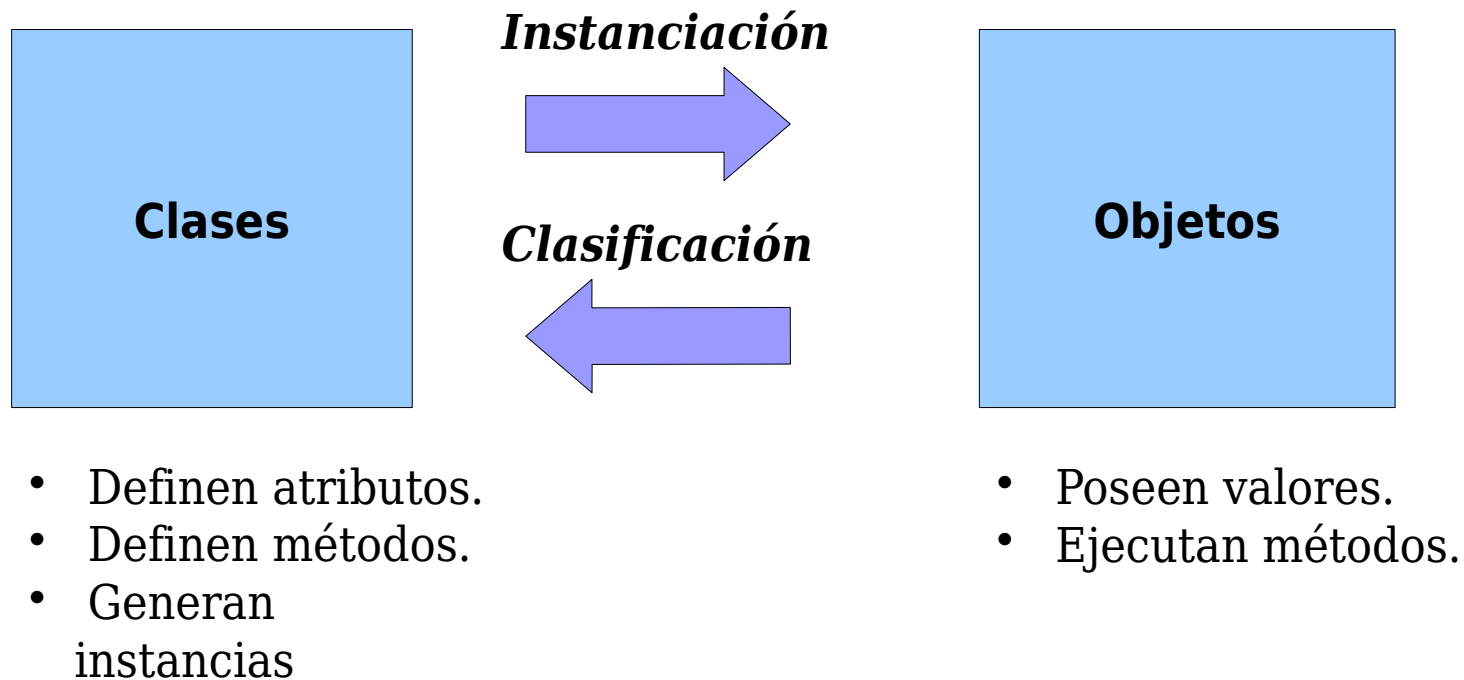


The screenshot shows a code editor with two tabs: 'faces-config.xml' and 'User.java'. The 'User.java' tab is active, displaying the following code:

```
package demo;  
  
/**  
 * Created by Red Hat Developer Studio  
 */  
public class User {  
  
    private String name;
```

Diferencias entre Clases y Objetos

- Las clases son ***definiciones estáticas*** que se pueden utilizar para entender a todos los objetos de una clase
- Los objetos son ***entidades dinámicas*** que existen en el mundo real.



Los atributos de un Objeto mantienen su estado

- Cada objeto conoce su estado actual.
- Un atributo es un fragmento de información que indica el estado de un objeto.



Birome



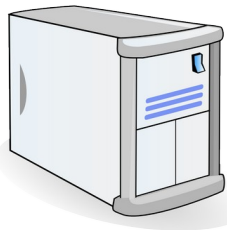
- Color
- Marca
- Cantidad de Tinta



Motocicleta



- Modelo
- Velocidad Máxima
- Cilindrada



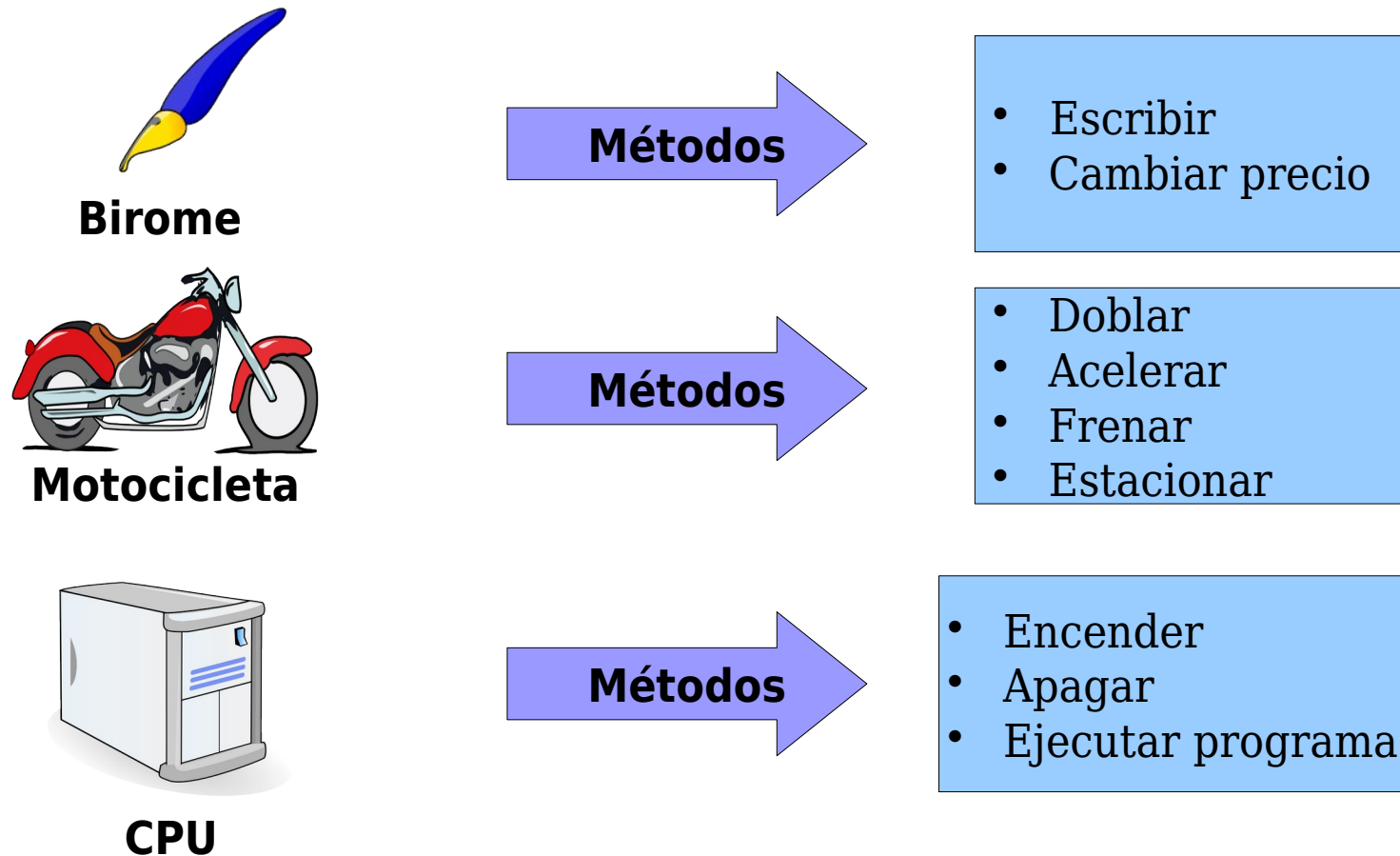
CPU



- Microprocesador
- Cantidad RAM
- Espacio en Disco

Los Objetos tienen comportamientos

- Un objeto existe para proporcionar un comportamiento (funcionalidad) al sistema.
- Cada comportamiento se denomina ***operación*** o ***método***.



Reglas de Nomenclatura

- Nombres de Clases

Persona
UsuarioInvitado
ClienteMoroso

- Nombres de Archivos

Persona.java
UsuarioInvitado.java
ClienteMoroso.java

- Nombres de Métodos

registrarLogin()
comprarProducto()
obtenerEdad()

- Nombres de Atributos

edad
comisionMaxima
cantidadVentas

Definición de Clases

- Utilizaremos la palabra reservada **class** para definir una clase.
-
- Generalmente la definición de una clase está formada por:

Modificador de Acceso



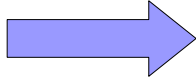
Especifica la disponibilidad de esa clase.

Campos de instancia



Definen los atributos de la clase.

Constructores



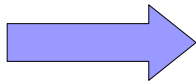
Métodos con el mismo nombre de la clase, controlan su estado inicial.

Métodos de Instancia



Definen las acciones que las instancias de la clase pueden realizar.

Campos de clase



Atributos compartidos por todas las instancias de la clase.

Métodos de clase



Métodos utilizados para controlar los atributos de clase.

Definición de Clases

Un ejemplo...

```
public class Usuario {
```



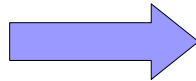
Definición de la clase

```
    private String nombre;  
    private String password;  
    public static int cantidadLogins;
```



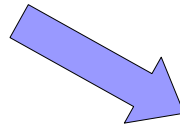
Variables de instancia y de clase

```
    public Usuario() {  
    }  
}
```

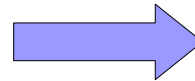


Constructor de la clase

```
    public String getNombre() {  
        return nombre;  
    }
```

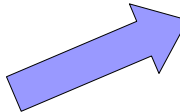


```
    public void setNombre(String nombre) {  
        this.nombre = nombre;  
    }
```



Métodos de Instancia

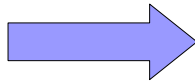
```
    public String getPassword() {  
        return password;  
    }
```



Definición de Métodos

- Siempre se especifican dentro de una clase.
- Generalmente la definición de un método está formada por:

Modificador de Acceso



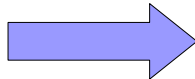
Especifica desde donde puede invocarse el método

Palabra clave static



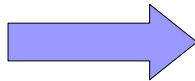
Especifica que el método es estático.

Tipo de Retorno



Especifica el tipo de valor que el mismo debe devolver.

Argumentos



Parámetros necesarios para que el método realice su función

Nombre

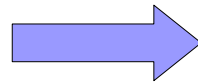


Nombre del método

Definición de Métodos

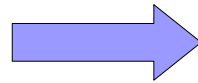
Algunos ejemplos...

```
public void setApellido(String apellido) {  
    this.apellido = apellido;  
}
```



Método para establecer una propiedad

```
public void mostrarNombre() {  
    String nombreCompleto = nombre + " , " + apellido;  
    System.out.println(nombreCompleto);  
}
```



Método sin retorno (void)

```
public static int getCantidadLogins() {  
    return cantidadLogins;  
}
```

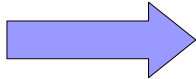


Método estático

Definición de Atributos

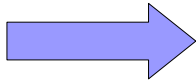
- Siempre se especifican dentro de una clase.
- Generalmente están definidos por:

Modificador de Acceso



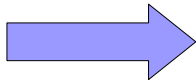
Especifica desde donde puede verse el atributo.

Palabra clave static



Especifica que el atributo es estático.

Tipo de dato



Especifica el tipo de dato del atributo.

Nombre

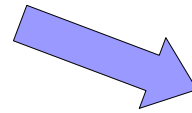


Nombre con el cual lo identificaremos.

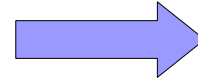
Definición de Atributos

Algunos ejemplos...

```
private long dni;
```

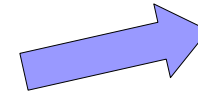


```
private String nombre;
```

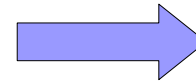


Atributos privados

```
private Date fechaNacimiento;
```



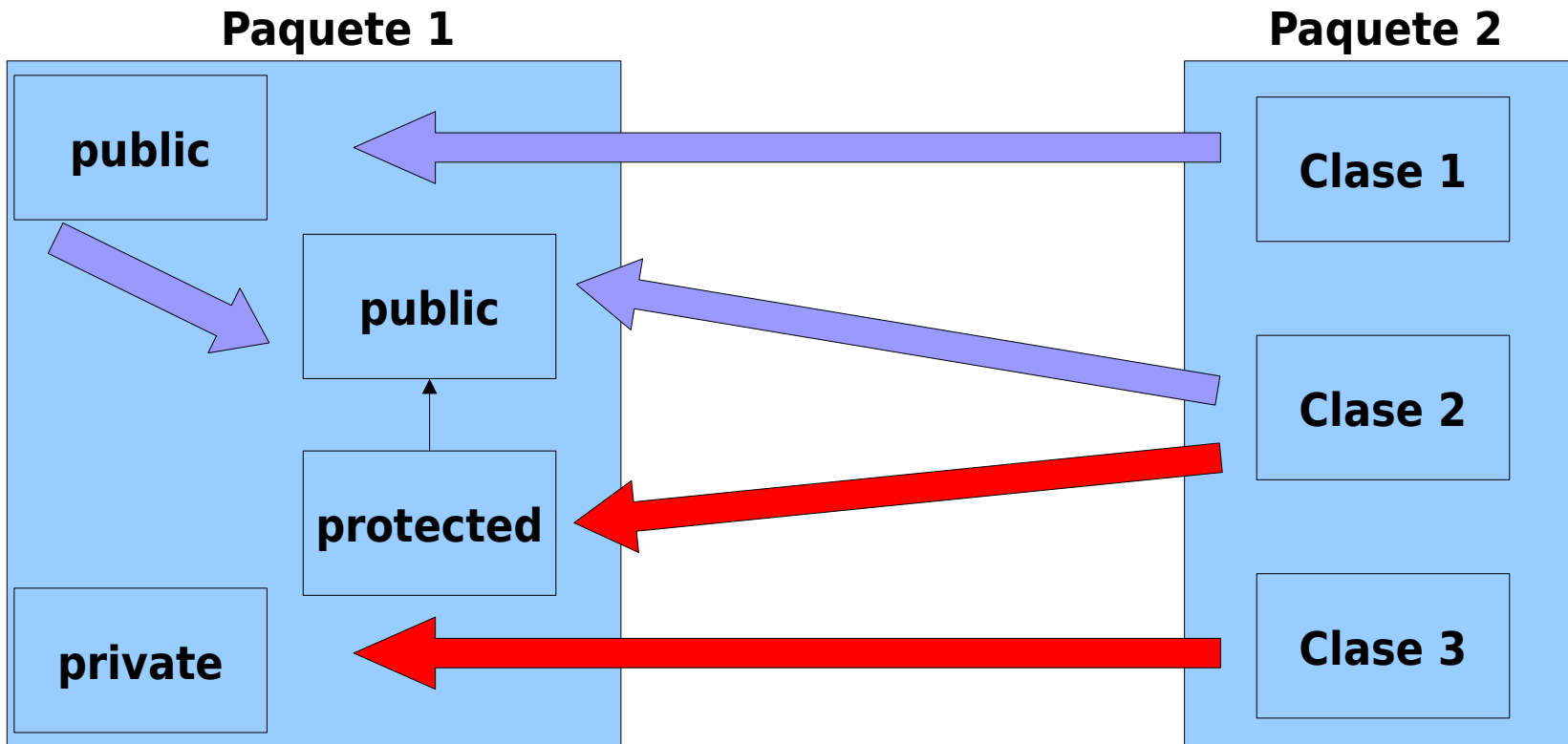
```
public static boolean cargaHabilitada = true;
```



Atributo público

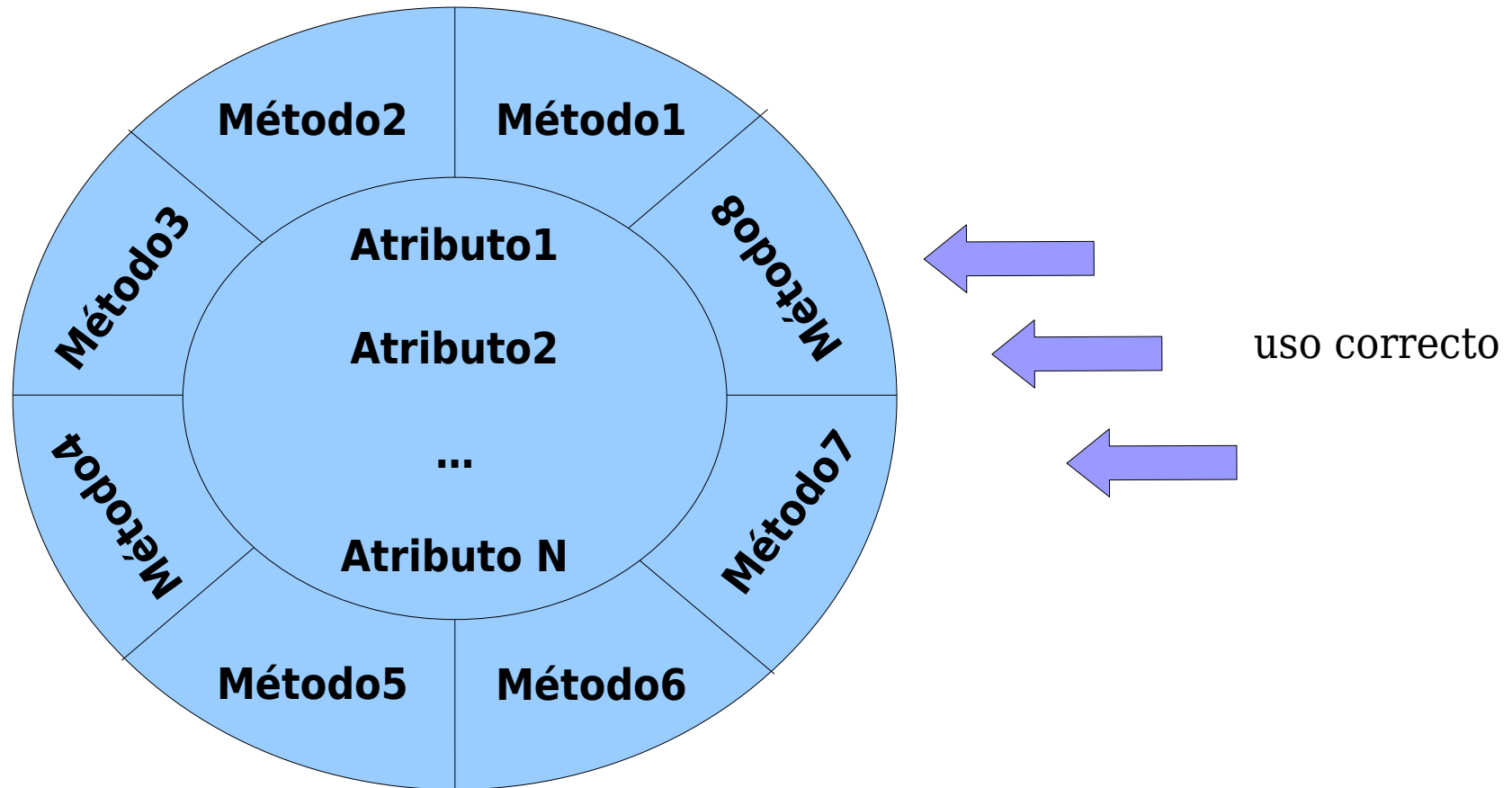
Modificadores de Acceso

- Controlan el acceso a los atributos y métodos.
- Para utilizar la encapsulación se recomienda utilizar variables privadas y métodos públicos.



Encapsulamiento

- Es una propiedad que asegura que la información de un módulo este oculta al exterior.
- Ocultando el estado de un objeto, solamente podremos modificar sus atributos mediante sus métodos.



Modificadores de Acceso

- Existen 3 tipos a definir, ***public*** - ***protected*** - ***private***.
- Si no especificamos ninguno se utiliza default.

<i>Accesible a</i>	<i>Public</i>	<i>Protected</i>	<i>Default</i>	<i>Private</i>
<i>Misma clase</i>	Sí	Sí	Sí	Sí
<i>Cualquier clase del mismo paquete</i>	Sí	Sí	Sí	No
<i>Clase hija de un paquete diferente</i>	Sí	Sí	No	No
<i>Cualquier otra</i>	Sí	No	No	No

Métodos get (obtener) y set (establecer)

Setters

- Cambian el valor de un atributo.
- Comienzan con *set* seguido del nombre del atributo.
- Ejemplos:
 - `void setApellido(String apellido) { ... };`
 - `void setEdad(Date edad) { ... };`
 - `void setNombre(String nombre) { ... };`

Getters

- Obtienen el valor de un atributo.
- Comienzan con *get* seguido del nombre del atributo.
- Ejemplos:
 - `String getApellido() { ... };`
 - `Date getEdad() { ... };`
 - `String getNombre() { ... };`

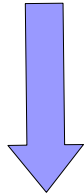
Uso de la referencia “this”

- Puede utilizarse dentro del cuerpo de un método de un objeto.
- Lo utilizamos para evitar la ambigüedad entre nombres de atributos o métodos.

```
public void setNombre(String nombre) {  
    this.nombre = nombre;  
}
```



Variable *nombre* utilizada como parámetro.



this.nombre especifica la propiedad nombre de la clase actual.

- También podemos utilizarlo para invocar a métodos o constructores.

```
String nom = this.getNombre();
```

Constructores

- Métodos con el mismo nombre de la clase.
- Se utilizan para controlar el estado inicial en el que se crea un objeto.
- No poseen valor de retorno (ni siquiera void).
- No se pueden invocar directamente como otro método.

```
public Usuario() {  
}
```

Constructor vacío

```
public Usuario(long dni, String nombre, Date fechaNacimiento) {  
    super();  
    this.dni = dni;  
    this.nombre = nombre;  
    this.fechaNacimiento = fechaNacimiento;  
}
```

Constructor con Parámetros

Creando de Objetos

Primeramente debemos crear una variable de referencia a ese objeto.

```
Usuario u;
```

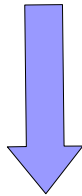
Luego debemos inicializar esa referencia utilizando el operador *new* y llamando al constructor correspondiente:

```
u = new Usuario();
```



Constructor vacío

```
Usuario u = new Usuario(30164258, "nflores", new Date("25/05/1983"));
```



Constructor con Parámetros

Utilizando de Objetos

Existen dos formas de usar un objeto:

- Manipulando sus atributos

```
u.dni = 34629273;
```

**No es recomendable,
rompe la encapsulación**

- Invocando sus métodos

```
long l = u.getDni();
```

Utilizamos el operador “.”

```
u.setNombre("invitado");
```